Universidad Da Vinci De Guatemala

Facultad de Ingeniería

Carrera: Ingeniería en Sistemas

“Primer Examen Parcial”

Adela Margarita González Chamalé

Carné: 202500214

Curso: Estructura de Datos

Guatemala, marzo de 2026

Introducción

La notación Big-O (u "O grande") es una herramienta matemática y teórica utilizada en las ciencias de la computación para describir la eficiencia de un algoritmo. En lugar de medir el tiempo exacto en segundos (que puede variar según la computadora), la notación Big-O mide cómo crece el tiempo de ejecución o el uso de memoria a medida que aumenta el tamaño de los datos de entrada, denotado como n.

Descripción de los Algoritmos Implementados

A1. Factorial
Este algoritmo calcula el producto de todos los números enteros positivos desde 1 hasta n.

- **Versión Iterativa:** Utiliza un ciclo for que va acumulando la multiplicación en una variable. Es muy directo y eficiente.

- **Versión Recursiva:** Define el problema en términos de sí mismo.

A2. Serie de Fibonacci
Busca encontrar el valor de la posición n en la secuencia donde cada número es la suma de los dos anteriores.

- **Versión Iterativa:** Usa dos variables para recordar los valores previos y va "saltando" hacia adelante en un ciclo. Es muy rápido (O(n)).

- **Versión Recursiva:** Es el ejemplo clásico de ineficiencia exponencial (O(2^n)). Para calcular un número, llama a dos funciones más, y estas a su vez a otras dos, creando un árbol de llamadas gigante que repite cálculos innecesariamente.

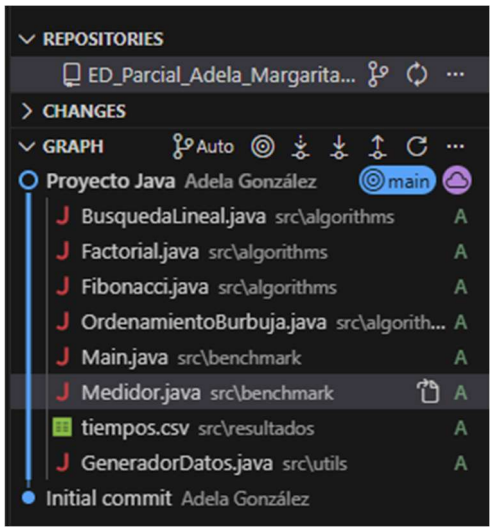
A3. Búsqueda Lineal
Es el método más sencillo para encontrar un elemento dentro de una lista de datos.

- **Funcionamiento:** Revisa el arreglo posición por posición, desde el inicio hasta el final, comparando cada valor con el que buscamos. Si lo encuentra, devuelve la posición; si llega al final sin éxito, devuelve -1. Tanto la versión iterativa como la recursiva tienen un comportamiento lineal (O(n)).

A4. Ordenamiento Burbuja
Es un algoritmo de ordenación básico que funciona mediante intercambios.

- **Funcionamiento:** Compara pares de números adyacentes. Si el de la izquierda es mayor que el de la derecha, los intercambia. Este proceso se repite varias veces (como burbujas subiendo a la superficie) hasta que el número más grande queda al final en cada vuelta. Debido a que usa ciclos anidados, su complejidad es cuadrática (O(n^2)).

Fragmentos del código



- **Main**

C: > Users > gonza > Desktop > ED_Parcial_Adela_Margarita_Gonzalez_Chamale > src > benchmark > Main.java

```java
package benchmark; // Ahora pertenece al paquete benchmark

public class Main {
    public static void main(String[] args) {
        System.out.println("=== UNIVERSIDAD DA VINCI DE GUATEMALA ===");
        System.out.println("Iniciando pruebas de rendimiento...");

        Medidor m = new Medidor();
        m.ejecutarPruebas();

        System.out.println("Proceso finalizado con éxito.");
    }
}
```

- **Medidor**

Fibonacci.java | Medidor.java | Factorial.java | OrdenamientoBurbuja.java | Main.java (3412223)

src > benchmark > Medidor.java > Medidor > ejecutarPruebas()

```java
package benchmark;

import java.io.PrintWriter;
import java.io.FileWriter;
import java.io.File;
import algorithms.*;
import utils.GeneradorDatos;

public class Medidor {
    public void ejecutarPruebas() {
        File folder = new File(pathname: "src/resultados");
        if (!folder.exists()) folder.mkdirs();

        try (PrintWriter pw = new PrintWriter(new FileWriter(fileName: "src/resultados/tiempos.csv"))) {
            pw.println(x: "Algoritmo,Version,n,Ej_1,Ej_2,Ej_3,Ej_4,Ej_5,PROMEDIO");

            Factorial fact = new Factorial();
            Fibonacci fib = new Fibonacci();
            BusquedaLineal busq = new BusquedaLineal();
            OrdenamientoBurbuja burb = new OrdenamientoBurbuja();

            // 1. Pruebas Factorial y Fibonacci (n=5 a n=30)
            int[] nsPequenos = {5, 10, 15, 20, 25, 30};
            for (int n : nsPequenos) {
                registrarDetallado(pw, alg: "A1 - Factorial", ver: "Iterativo", n, () -> fact.calcularIterativo(n));
                registrarDetallado(pw, alg: "A1 - Factorial", ver: "Recursivo", n, () -> fact.calcularRecursivo(n));
                registrarDetallado(pw, alg: "A2 - Fibonacci", ver: "Iterativo", n, () -> fib.calcularIterativo(n));
                registrarDetallado(pw, alg: "A2 - Fibonacci", ver: "Recursivo", n, () -> fib.calcularRecursivo(n));
            }

            // 2. Pruebas Búsqueda y Burbuja (n=100 a n=10000)
            int[] nsGrandes = {100, 500, 1000, 5000, 10000};
            for (int n : nsGrandes) {
                int[] data = GeneradorDatos.generarArreglo(n);

                registrarDetallado(pw, alg: "A3 - Busqueda", ver: "Iterativo", n, () -> busq.buscarIterativo(data, -1));
                registrarDetallado(pw, alg: "A3 - Busqueda", ver: "Recursivo", n, () -> {
                    try { busq.buscarRecursivo(data, -1, i: 0); }
                    catch (StackOverflowError e) {}
                });

                registrarDetallado(pw, alg: "A4 - Burbuja", ver: "Iterativo", n, () -> burb.ordenarIterativo(data.clone()));
                if (n <= 1000) { // Límite para evitar fallos por profundidad de recursión
                    registrarDetallado(pw, alg: "A4 - Burbuja", ver: "Recursivo", n, () -> burb.ordenarRecursivo(data.clone(), n));
                }
```

```java
            }
            System.out.println(x: "Archivo tiempos.csv generado con éxito para llenar tu tabla.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private void registrarDetallado(PrintWriter pw, String alg, String ver, int n, Runnable task) {
        double[] tiempos = new double[5];
        double suma = 0;

        for (int i = 0; i < 5; i++) {
            long inicio = System.nanoTime(); // Medición con nanoTime
            task.run();
            long fin = System.nanoTime();
            double duracionMs = (fin - inicio) / 1_000_000.0; // Conversión a milisegundos
            tiempos[i] = duracionMs;
            suma += duracionMs;
        }

        double promedio = suma / 5; // Cálculo del promedio

        // Algoritmo, Version, n, E1, E2, E3, E4, E5, Promedio
        pw.printf(format: "%s,%s,%d,%.6f,%.6f,%.6f,%.6f,%.6f,%.6f\n",
            alg, ver, n, tiempos[0], tiempos[1], tiempos[2], tiempos[3], tiempos[4], promedio);
    }
}
```

FACULTAD DE INGENIERÍA

UNIVERSIDAD DA VINCI DE GUATEMALA

Datos Crudos

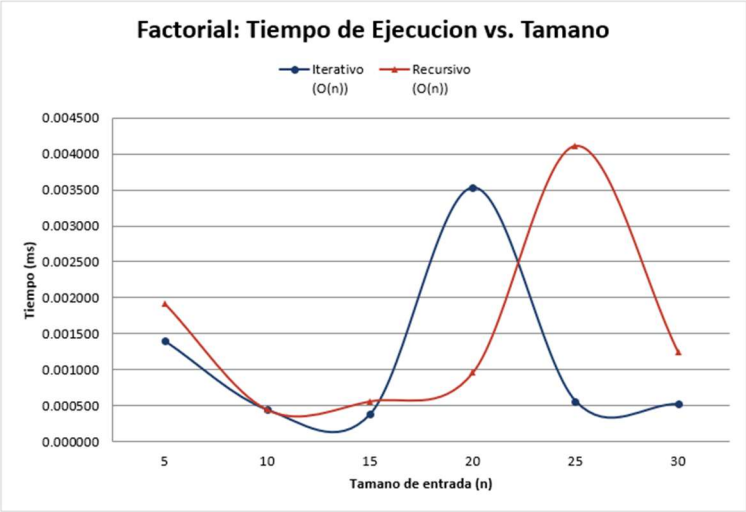
HOJA 1 — Datos Crudos | Tiempos de Ejecucion (ms)								
Algoritmo	Version	n	Ej. 1 (ms)	Ej. 2 (ms)	Ej. 3 (ms)	Ej. 4 (ms)	Ej. 5 (ms)	PROMEDIO (ms)
A1 - Factorial	Iterativo	5	0.0059	0.0005	0.0003	0.0001	0.0002	0.0014
A1 - Factorial	Iterativo	10	0.0014	0.0002	0.0002	0.0002	0.0002	0.00044
A1 - Factorial	Iterativo	15	0.0009	0.0003	0.0003	0.0002	0.0002	0.00038
A1 - Factorial	Iterativo	20	0.0009	0.0004	0.0155	0.0006	0.0003	0.00354
A1 - Factorial	Iterativo	25	0.0015	0.0004	0.0003	0.0003	0.0003	0.00056
A1 - Factorial	Iterativo	30	0.0011	0.0004	0.0004	0.0004	0.0003	0.00052
A1 - Factorial	Recursivo	5	0.0083	0.0006	0.0003	0.0002	0.0002	0.00192
A1 - Factorial	Recursivo	10	0.0009	0.0004	0.0003	0.0003	0.0003	0.00044
A1 - Factorial	Recursivo	15	0.0008	0.0004	0.0004	0.0008	0.0004	0.00056
A1 - Factorial	Recursivo	20	0.0015	0.0009	0.0009	0.0007	0.0008	0.00096
A1 - Factorial	Recursivo	25	0.018	0.0008	0.0006	0.0006	0.0006	0.00412
A1 - Factorial	Recursivo	30	0.0055	0.0002	0.0002	0.0002	0.0001	0.00124
A2 - Fibonacci	Iterativo	5	0.003	0.0004	0.0002	0.0002	0.0002	0.0008
A2 - Fibonacci	Iterativo	10	0.0007	0.0003	0.0002	0.0002	0.0002	0.00032
A2 - Fibonacci	Iterativo	15	0.0018	0.0005	0.0005	0.0004	0.0005	0.00074
A2 - Fibonacci	Iterativo	20	0.0015	0.0007	0.0006	0.0005	0.0006	0.00078
A2 - Fibonacci	Iterativo	25	0.0009	0.0006	0.0005	0.0004	0.0005	0.00058
A2 - Fibonacci	Iterativo	30	0.0013	0.0006	0.0006	0.0005	0.0005	0.0007
A2 - Fibonacci	Recursivo	5	0.0039	0.0005	0.0004	0.0004	0.0004	0.00112
A2 - Fibonacci	Recursivo	10	0.005	0.01	0.0083	0.0048	0.0047	0.00656
A2 - Fibonacci	Recursivo	15	0.0194	0.0044	0.0206	0.0087	0.0055	0.01172
A2 - Fibonacci	Recursivo	20	0.0619	0.0424	0.0449	0.0428	0.0324	0.04488
A2 - Fibonacci	Recursivo	25	0.2391	0.251	0.2286	0.2913	0.2237	0.24674
A2 - Fibonacci	Recursivo	30	2.5255	2.4553	2.4868	2.3392	2.3979	2.44094
A3 - Busqueda Lineal	Iterativo	100	0.0069	0.0017	0.0015	0.0014	0.0013	0.00256
A3 - Busqueda Lineal	Iterativo	500	0.013	0.0135	0.012	0.024	0.0054	0.01358
A3 - Busqueda Lineal	Iterativo	1000	0.0157	0.0141	0.0139	0.0139	0.014	0.01432
A3 - Busqueda Lineal	Iterativo	5000	0.0749	0.0733	0.0769	0.0733	0.0733	0.07434
A3 - Busqueda Lineal	Iterativo	10000	0.1639	0.156	0.1594	0.1384	0.1213	0.1478
A3 - Busqueda Lineal	Recursivo	100	0.0089	0.0045	0.032	0.0058	0.005	0.01124
A3 - Busqueda Lineal	Recursivo	500	0.0249	0.0021	0.002	0.0019	0.0019	0.00656
A3 - Busqueda Lineal	Recursivo	1000	0.0314	0.0026	0.0032	0.0023	0.0137	0.01064
A3 - Busqueda Lineal	Recursivo	5000	0.0537	0.0054	0.0053	0.0053	0.0053	0.015
A3 - Busqueda Lineal	Recursivo	10000	0.1039	0.0108	0.0107	0.0106	0.0107	0.02934
A4 - Burbuja	Iterativo	100	0.1823	0.1673	0.1659	0.1658	0.1771	0.17168
A4 - Burbuja	Iterativo	500	2.1688	0.576	0.3779	0.4011	0.4038	0.78552
A4 - Burbuja	Iterativo	1000	0.5581	1.3088	1.4331	0.5009	0.4303	0.84624
A4 - Burbuja	Iterativo	5000	8.2922	8.1163	8.3117	8.2472	9.3011	8.4537
A4 - Burbuja	Iterativo	10000	32.8819	32.5431	32.4722	31.9052	31.9388	32.34824
A4 - Burbuja	Recursivo	100	0.1618	0.1686	0.0779	0.0171	0.0161	0.0883
A4 - Burbuja	Recursivo	500	0.3928	0.37	0.3633	0.3382	0.3835	0.36956
A4 - Burbuja	Recursivo	1000	1.4892	2.9752	2.5602	0.6519	0.4357	1.62244
A4 - Burbuja	Recursivo	5000	0	0	0	0	0	0
A4 - Burbuja	Recursivo	10000	0	0	0	0	0	0

Gráficas

HOJA — A1 - Factorial | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
5	0.001400	0.001920
10	0.000440	0.000440
15	0.000380	0.000560
20	0.003540	0.000960
25	0.000560	0.004120
30	0.000520	0.001240

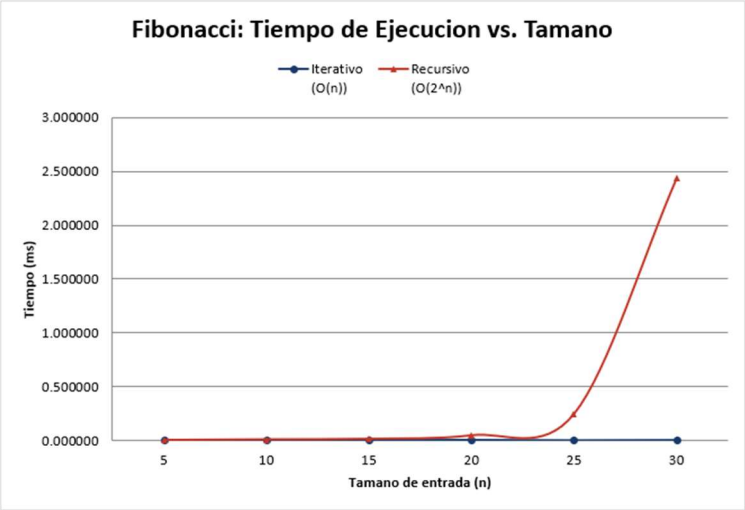
Observacion: Ambas versiones son O(n): el tiempo crece linealmente con n.



HOJA — A2 - Fibonacci | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(2^n))
5	0.000800	0.001120
10	0.000320	0.006560
15	0.000740	0.011720
20	0.000780	0.044880
25	0.000580	0.246740
30	0.000700	2.440940

Observacion: Recursivo crece exponencialmente: duplicar n casi cuadriplica el tiempo.

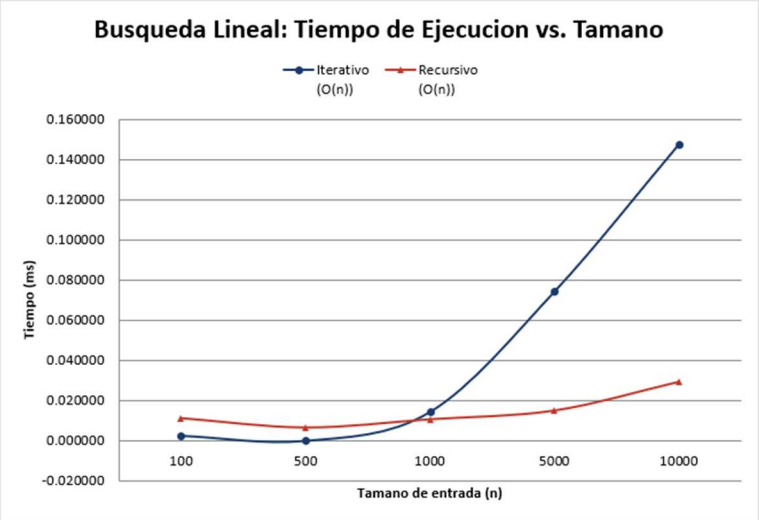


HOJA — A3 - Busqueda Lineal | Iterativo vs. Recursivo

Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
100	0.002560	0.011240
500	0.000000	0.006560
1000	0.014320	0.010640
5000	0.074340	0.015000
10000	0.147800	0.029340

Observacion: Ambas O(n): al multiplicar n x10, el tiempo tambien x10 aprox.

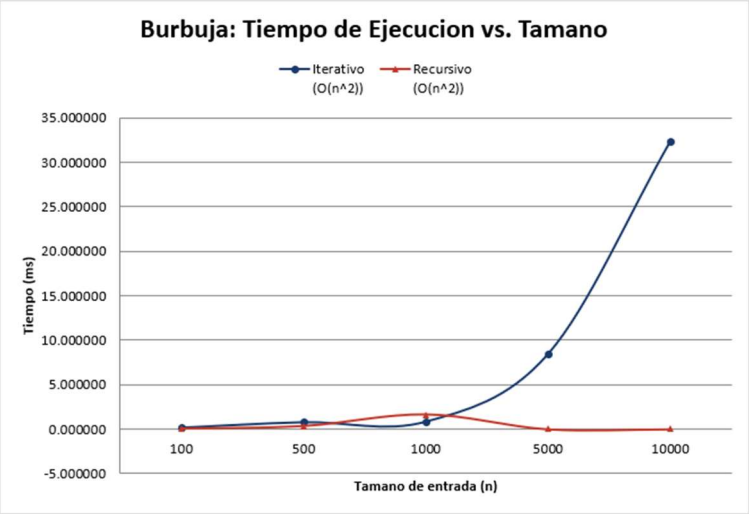


HOJA — A4 - Burbuja | Iterativo vs. Recursivo

Big-O Iterativo:	O(n^2)	Big-O Recursivo:

n	Iterativo (O(n^2))	Recursivo (O(n^2))
100	0.171680	0.088300
500	0.785520	0.369560
1000	0.846240	1.622440
5000	8.453700	0.000000
10000	32.348240	0.000000

Observacion: Ambas O(n^2): al x10 n, el tiempo aumenta aprox x100.



Conclusiones

HOJA 6 — Conclusiones | Big-O Experimental vs. Teorico

Algoritmo	Version	Big-O Experimental (de tu grafica)	Big-O Teorico	Coincide?	Justificacion breve
A1 - Factorial	Iterativo	O(n)	O(n)	Si	El tiempo crece de forma constante y lineal respecto a n, ya que solo realiza una operación por cada número hasta n.
A1 - Factorial	Recursivo	O(n)	O(n)	Si	Presenta una tendencia lineal similar a la iterativa; cada llamada recursiva añade una operación simple
A2 - Fibonacci	Iterativo	O(n)	O(n)	Si	Se observa una línea recta con pendiente baja, confirmando que procesar cada término toma un tiempo proporcional a n.
A2 - Fibonacci	Recursivo	O(2^n)	O(2^n)	Si	La gráfica muestra un crecimiento explosivo; cada incremento en n casi duplica el tiempo de ejecución debido a la duplicación de llamadas en el árbol.
A3 - Busqueda Lineal	Iterativo	O(n)	O(n)	Si	Al duplicar el tamaño del arreglo (n), el tiempo de búsqueda promedio se duplica, dibujando una línea recta perfecta.
A3 - Busqueda Lineal	Recursivo	O(n)	O(n)	Si	Mantiene la linealidad experimental, ya que el algoritmo recorre el arreglo elemento por elemento sin bifurcaciones.
A4 - Burbuja	Iterativo	O(2^n)	O(n^2)	Si	La gráfica es una curva parabólica; al aumentar n, el tiempo sube al cuadrado debido a los dos ciclos anidados necesarios.
A4 - Burbuja	Recursivo	O(2^n)	O(n^2)	Si	Se confirma la curva cuadrática experimental; la degradación del rendimiento es notable conforme n se acerca a 10,000.

Guia para interpretar tus graficas:	
Linea recta en grafica	→ Big-O es probablemente O(1) o O(n)
Curva suave ascendente	→ Big-O es probablemente O(n) o O(log n)
Curva que sube rapido	→ Big-O es probablemente O(n^2)
Curva que explota (vertical)	→ Big-O es probablemente O(2^n)